JOBSHEET 18

OPTIMASI PERFORMA APLIKASI MENGGUNAKAN FITUR NEXT.JS

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

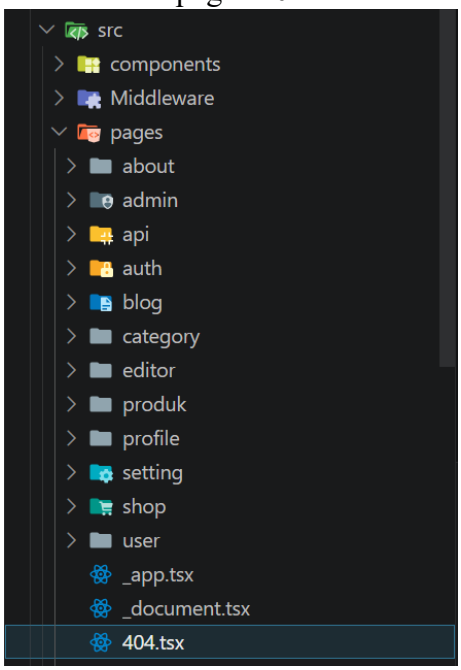
Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_18

PRAKTIKUM 1 – Image Optimization

A. Optimasi Gambar Lokal (Public Folder)

- Studi Kasus:
- Mengganti tag <img> pada halaman 404 dengan next/image. Langkah:
- Buka file src/pages/404.tsx



- Modifikasi line 7 menjadi line 8-11

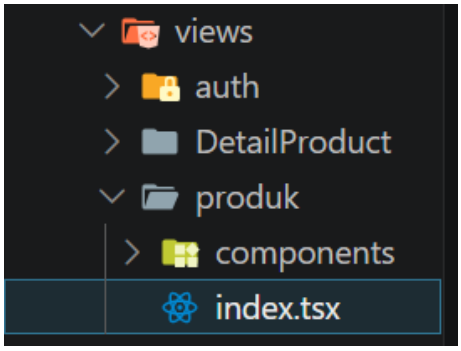
```
import Head from "next/image";
import styles from "@/styles/404.module.scss";
// TUGAS 3: KEMBALI KE HOME
import Link from "next/link";
import Image from "next/image";

const Custom404 = () => {
  return (
    <>
      {/* <Head>
        <title>404 - Halaman Tidak Ditemukan</title>
        <meta
          name="description"
          content="Halaman yang Anda cari tidak tersedia."
        />
      </Head> */}

      <div className={styles.error}>
        {/* <h1 className={styles.error_title}>
          404 - Halaman Tidak Ditemukan
        </h1>

        <p className={styles.error_desc}>
          Maaf, halaman yang Anda cari mungkin telah dipindahkan atau tidak tersedia.
        </p> */}

        {/* <img
          src="/page-not-found.png"
          alt="Halaman tidak ditemukan"
          className={styles.error_image}
        /> */}

          <Image
          src="/page-not-found.png"
          alt="404"
          width={400}
          height={200}
          className={styles.error_image}
          />

          <h1>404 - Halaman Tidak Ditemukan</h1>
          <p>Maaf, halaman yang Anda cari mungkin telah dipindahkan atau tidak tersedia.</p>


    {/* TUGAS 3: TOMBOL KEMBALI KE HOME */}
        {/* <Link href="/">
        <button className={styles.back_button}>
            Kembali ke Home
        </button>
        </Link> */}

      </div>
    </>
  );
};

export default Custom404;
```

Hasil:

- Warning hilang
- Image dioptimasi otomatis
- Mengurangi bandwidth
- Mendukung lazy loading otomatis

B. Optimasi Gambar Remote (External URL)

- Buka file views/product/index.tsx



- Modifikasi file index.tsx

```
my-app-18 > src > views > produk > index.tsx > ...
1  import styles from "../../styles/product.module.scss";
2  import Link from "next/link";
3  import Image from "next/image";
```

```
my-app-18 > src > views > produk > index.tsx > Tampilan
13   const TampilanProduk = ({ products = [] }:
22           {products.map((product: ProductT
23             <Link href={`/produk/${product
24               <div
25                 // key={product.id}
26                 // className={styles.produ
27               >
28                 <div className={styles.pro
29                   <Image
30                     src={product.image}
31                     alt={product.name}
32                     width={200}
33                     height={200}
34                   />
```

Note : dikarenakan gambar diambil dari url tertentu maka konfigurasi berbeda

- Buka file next.config.js

```jsx
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "assets.adidas.com",
        port: "",
        pathname: "/**",
      },
      {
        protocol: "https",
        hostname: "lh3.googleusercontent.com",
        pathname: "/**",
      },
      {
        protocol: "https",
        hostname: "avatars.githubusercontent.com",
        pathname: "/**",
      },
    ],
  },
};

module.exports = nextConfig;
```

Hasil:

- Gambar di-proxy melalui /_next/image
- Performa lebih optimal
- Kompresi otomatis

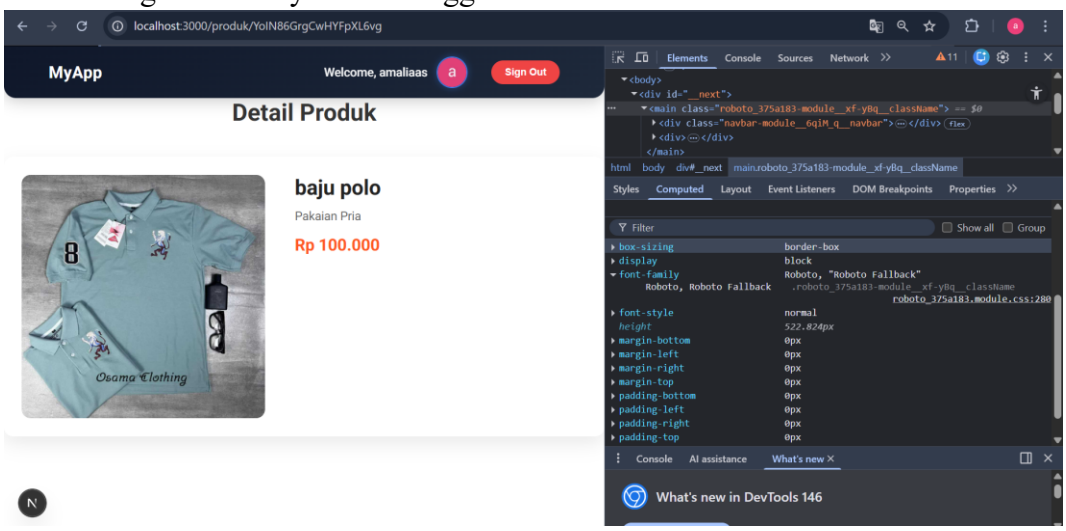
PRAKTIKUM 2 – Font Optimization

A. Menggunakan next/font

- Buka file index.tsx pada folder Appshell/index.tsx dan modifkasi

```python
import Navbar from "@/components/navbar";
import { useRouter } from "next/router";
import { Roboto } from "next/font/google";

// Daftar path yang tidak menampilkan Navbar
const disableNavbar = ["/auth/login", "/auth/register", "/404"];
type AppShellProps = {
    children: React.ReactNode;
}

const roboto = Roboto({
    subsets: ["latin"],
    weight: ["400", "500", "700"],
});

const AppShell = (props:AppShellProps) => {
    const { children } = props;
    const { pathname } = useRouter();
    const router = useRouter();
    console.log(router);

    return (
        <main className={roboto.className}>
            {!disableNavbar.includes(pathname) && <Navbar />}
        {children}
        {/* <footer style={{ marginTop: "20px" }}>
            <hr />
            <p> © 2026 - Praktikum Next.js</p>
        </footer> */}
        </main>
    );
    };

export default AppShell;
```

- Jalankan browser localhost:3000/produk maka font akan berubah menjadi roboto untuk mengecek fontnya bisa menggunakan extension FontFinder



Hasil:

- Tidak perlu load dari CDN manual
- Tidak blocking render
- Performance meningkat
- Tidak terjadi FOUT (Flash of Unstyled Text)

PRAKTIKUM 3 – Script Optimization

B. Menggunakan next/script

- Buka file index.tsx pada folder layouts/Navbar dan modifikasi

```
1  import styles from './navbar.module.css';
2  import { signIn, signOut, useSession } from 'next-auth/react';
3  import Image from 'next/image';
4  import Script from 'next/dist/client/script';
5
6  const Navbar = () => {
7    const { data }: any = useSession();
8
9    return (
10     <div className={styles.navbar}>
11
12       <div className={styles.navbar__brand} id="title"></div>
13       <Script id="title-script" strategy='lazyOnload'>
14         {`document.getElementById('title').textContent = 'MyApp';`}
15       </Script>
```

- Pada kasus diatas kita merubah line 11 menggunakan model Typescript dapatterlihat ketika kita refresh web produk tulisan myApp akan terlihat berkedip
- Perbedaan mendasar antara Line 11-13 dan Line 15-18 pada file index.tsx Anda terletak pada metode rendering teks dan interaksi dengan DOM (Document Object Model). Berikut adalah rincian perbedaannya:
  I. Metode Rendering
  a) Line 11-13 (Standard React/JSX): Ini adalah cara standar React. Teks "MyApp" ditulis langsung di dalam tag div. React akan langsung merender teks ini ke dalam HTML saat komponen dimuat.
  b) Line 15-18 (Next.js Script & Manual DOM): Menggunakan komponen <Script> dari Next.js dengan strategi lazyOnload. Teks tidak ada di dalam HTML saat awal dimuat, melainkan baru "disuntikkan" secara manual menggunakan perintah JavaScript document.getElementById('title').innerHTML = 'MyApp'; setelah script tersebut diunduh di latar belakang.

  II. Performa dan SEO

a) Line 11-13: Sangat baik untuk SEO karena teks "MyApp" langsung terbaca oleh robot pencari (crawler) dalam kode sumber HTML.
b) Line 15-18: Kurang baik untuk SEO untuk konten penting karena teks baru muncul setelah JavaScript dijalankan. Strategi lazyOnload berarti script ini dijalankan paling akhir setelah semua sumber daya utama selesai dimuat, sehingga mungkin ada jeda waktu (delay) sebelum teks muncul di layar.

III. Keamanan (XSS)

a) Line 11-13: Aman karena React secara otomatis melakukan escape pada string untuk mencegah serangan Cross-Site Scripting (XSS).
b) Line 15-18: Memiliki risiko keamanan lebih tinggi karena menggunakan .innerHTML. Jika data yang dimasukkan berasal dari input user, ini bisa dimanfaatkan untuk menyuntikkan skrip berbahaya.

C. Strategi Script

Strategy	Fungsi
beforeInteractive	Sebelum halaman interaktif
afterInteractive	Setelah halaman interaktif
lazyOnload	Setelah semua selesai
worker	Web worker

Hasil:

- Script tidak blocking
- Cocok untuk Google Analytics
- Performa lebih ringan

PRAKTIKUM 4 – Optimasi Avatar dengan next/image

- Buka file index.tsx pada folder layouts/navbar dan modifikasi :

```
1   <div className={styles.navbar__user}>
2      Welcome, {data.user?.fullname || data.user?.name}
3      {data.user.image && (
4        <Image
5          width={50}
6          height={50}
7          src={data.user.image}
8          alt={data.user.fullname}
9          className={styles.navbar__user__image}
10       />
11     )}
```

- Tambahkan hostname Google:

```javascript
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "assets.adidas.com",
        port: "",
        pathname: "/**",
      },
      {
        protocol: "https",
        hostname: "lh3.googleusercontent.com",
        pathname: "/**",
      },
      {
        protocol: "https",
        hostname: "avatars.githubusercontent.com",
        pathname: "/**",
      },
      // gambar dari Shopee
      {
        protocol: "https",
        hostname: "down-id.img.susercontent.com",
        pathname: "/**",
      },
      // gambar dari Pinterest
      {
        protocol: "https",
        hostname: "i.pinimg.com",
        pathname: "/**",
      },
      //gambar dari Nike
      {
        protocol: "https",
        hostname: "static.nike.com",
        pathname: "/**",
      },
    ],
  },
};

module.exports = nextConfig;
```

Tugas Praktikum

1. Optimasi semua image di project menggunakan next/image
   src/pages/404.tsx

```
import styles from "@/styles/404.module.scss";
import Link from "next/link";
import Image from "next/image";

const Custom404 = () => {
  return (
    <>
      {/* <Head>
        <title>404 - Halaman Tidak Ditemukan</title>
        <meta
          name="description"
          content="Halaman yang Anda cari tidak tersedia."
        />
      </Head> */}

      <div className={styles.error}>
        {/* <h1 className={styles.error_title}>
          404 - Halaman Tidak Ditemukan
        </h1>

        <p className={styles.error_desc}>
          Maaf, halaman yang Anda cari mungkin telah dipindahkan atau tidak tersedia.
        </p> */}

        {/* <img
          src="/page-not-found.png"
          alt="Halaman tidak ditemukan"
          className={styles.error_image}
        /> */}

          <Image
          src="/page-not-found.png"
          alt="404"
          width={400}
          height={200}
          className={styles.error_image}
          />

          <h1>404 - Halaman Tidak Ditemukan</h1>
          <p>Maaf, halaman yang Anda cari mungkin telah dipindahkan atau tidak tersedia.</p>


    {/* TUGAS 3: TOMBOL KEMBALI KE HOME */}
        {/* <Link href="/">
        <button className={styles.back_button}>
            Kembali ke Home
        </button>
        </Link> */}

      </div>
    </>
  );
};

export default Custom404;
```

   src/views/produk/index.tsx

```
<div className={styles.produk__content__item__image}>
  <Image
    src={product.image}
    alt={product.name}
    width={200}
    height={200}
  />
</div>
```

src/components/navbar/index.tsx

```
<Image
  width={50}
  height={50}
  src={data.user.image}
  alt="avatar"
  className={styles.navbar__user__image}
/>
```

2. Gunakan minimal 1 font dari next/font
   Src/components/Appshell/index.tsx

```
import Navbar from "@/components/navbar";
import { useRouter } from "next/router";
import { Roboto } from "next/font/google";

// Daftar path yang tidak menampilkan Navbar
const disableNavbar = ["/auth/login", "/auth/register", "/404"];
type AppShellProps = {
    children: React.ReactNode;
}

const roboto = Roboto({
    subsets: ["latin"],
    weight: ["400", "500", "700"],
});

const AppShell = (props:AppShellProps) => {
    const { children } = props;
    const { pathname } = useRouter();
    const router = useRouter();
    console.log(router);

    return (
        <main className={roboto.className}>
            {!disableNavbar.includes(pathname) && <Navbar />}
        {children}
        {/* <footer style={{ marginTop: "20px" }}>
            <hr />
            <p> © 2026 - Praktikum Next.js</p>
        </footer> */}
        </main>
    );
    };

export default AppShell;
```

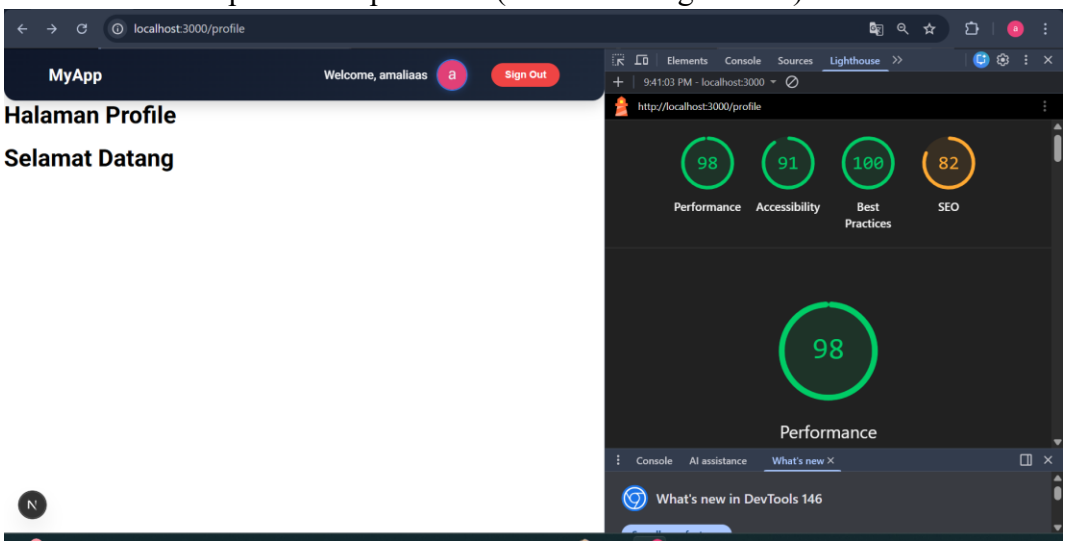
3. Tambahkan script Google Analytics menggunakan next/script
   Src/components/navbar/index.tsx

```python
import styles from './navbar.module.css';
import { signIn, signOut, useSession } from 'next-auth/react';
import Image from 'next/image';
import Script from 'next/dist/client/script';

const Navbar = () => {
  const { data }: any = useSession();

  return (
    <div className={styles.navbar}>

      <div className={styles.navbar__brand} id="title"></div>
      <Script id="title-script" strategy='lazyOnload'>
        {`document.getElementById('title').textContent = 'MyApp';`}
      </Script>

            {/* Google Analytics menggunakan next/script */}
      <Script
        src="https://www.googletagmanager.com/gtag/js?id=GA_MEASUREMENT_ID"
        strategy="lazyOnload"
      />
      <Script id="google-analytics" strategy="lazyOnload">
        {`
          window.dataLayer = window.dataLayer || [];
          function gtag(){dataLayer.push(arguments);}
          gtag('js', new Date());
          gtag('config', 'GA_MEASUREMENT_ID');
        `}
      </Script>

      <div className={styles.navbar__right}>
        {data ? (
          <>
            <div className={styles.navbar__user}>
              Welcome, {data.user?.fullname || data.user?.name}
              {data.user.image && (
                <Image
                  width={50}
                  height={50}
                  src={data.user.image}
                  alt="avatar"
                  className={styles.navbar__user__image}
                />
              )}
            </div>
            <button
              className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`}
              onClick={() => signOut({ callbackUrl: "/" })}
            >
              Sign Out
            </button>
          </>
        ) : (
          <button
            className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`}
            onClick={() => signIn(undefined, { callbackUrl: "/profile" })}
          >
            Sign In
          </button>
        )}
      </div>

    </div>
  );
};

export default Navbar;
```

4. Terapkan dynamic import pada minimal 1 komponen
   Dynamic import saya terapkan di Appshell/index.tsx untuk komponen Navbar, karena Navbar tidak perlu dimuat saat halaman pertama kali render

```python
import { useRouter } from "next/router";
import { Roboto } from "next/font/google";
import dynamic from "next/dynamic";

// Navbar dimuat secara dynamic (lazy loading)
// artinya Navbar hanya dimuat saat dibutuhkan, bukan saat halaman pertama render
const Navbar = dynamic(() => import("@/components/navbar"), {
  ssr: false, // tidak dirender di server, hanya di client
  loading: () => <div>Loading navbar...</div>, // ← tampilan saat Navbar belum dimuat
});

const disableNavbar = ["/auth/login", "/auth/register", "/404"];

type AppShellProps = {
  children: React.ReactNode;
};

const roboto = Roboto({
  subsets: ["latin"],
  weight: ["400", "500", "700"],
});

const AppShell = (props: AppShellProps) => {
  const { children } = props;
  const { pathname } = useRouter();

  return (
    <main className={roboto.className}>
      {!disableNavbar.includes(pathname) && <Navbar />}
      {children}
    </main>
  );
};

export default AppShell;
```

5. Dokumentasikan perubahan performa (screenshot Lighthouse)



Refleksi & Diskusi

1. Mengapa <img> biasa tidak optimal?
   - Tag `<img>` tidak memiliki fitur optimasi otomatis seperti kompresi, pengaturan ukuran, dan lazy loading. Akibatnya, gambar yang ditampilkan cenderung lebih besar dan memerlukan waktu loading lebih lama. Hal ini dapat memperlambat performa halaman, terutama jika jumlah gambar banyak atau ukuran gambar besar.
2. Apa perbedaan font CDN dan next/font?
   - Font CDN mengambil file font dari server eksternal sehingga membutuhkan koneksi tambahan dan dapat menyebabkan keterlambatan saat render. Sedangkan `next/font` mengelola font secara lokal di aplikasi sehingga proses loading lebih cepat, tidak bergantung pada koneksi eksternal, dan dapat menghindari efek FOUT (Flash of Unstyled Text).
3. Mengapa script bisa membuat website lambat?
   - Script dapat memperlambat website karena bersifat blocking, yaitu menghentikan proses render halaman sampai script selesai dimuat dan dijalankan. Jika jumlah script banyak atau ukurannya besar, maka waktu loading halaman akan semakin lama dan mempengaruhi performa secara keseluruhan.
4. Kapan harus menggunakan dynamic import?
   - Dynamic import digunakan ketika komponen tidak perlu dimuat saat halaman pertama kali dibuka. Biasanya digunakan pada komponen yang berat, jarang digunakan, atau hanya muncul pada kondisi tertentu seperti modal, chart, atau halaman tertentu. Tujuannya untuk mengurangi beban loading awal.
5. Apa dampak bundle size terhadap UX?
   - Bundle size yang besar akan membuat waktu loading menjadi lebih lama karena browser harus mengunduh file yang lebih banyak. Hal ini dapat menurunkan kenyamanan pengguna karena halaman terasa lambat. Sebaliknya, bundle size yang kecil akan mempercepat loading dan meningkatkan pengalaman pengguna (UX).